

Sistema basado en el microprocesador Z80 con memoria, interfaz PPI y programación en ensamblador

Romero Primitivo Alexis Aldair

Universidad Nacional Autónoma de México, Facultad de Estudios Superiores Cuautitlán

21 de septiembre de 2026

Resumen

En esta actividad se diseñó un sistema basado en el microprocesador Z80, integrado por memoria de código, memoria de datos e interfaces de entrada/salida. El sistema dispone de 16 KB destinados al almacenamiento de código, 16 KB de memoria de datos y seis puertos de entrada/salida implementados mediante dos interfaces programables 8255. Se estableció un mapa de direccionamiento en el cual la memoria de código inicia en 0000H, la memoria de datos en F800H y los puertos de entrada/salida a partir de 40H. El diagrama electrónico se desarrolló en KiCad Schematic Capture empleando una EEPROM 28C256, dos memorias SRAM 6264, dos interfaces 8255 y la lógica necesaria para la decodificación de direcciones y selección de dispositivos. Adicionalmente, se desarrolló un programa en lenguaje ensamblador para solicitar el nombre y apellidos del usuario, validar los caracteres introducidos, almacenar la información y determinar la cantidad de letras sin considerar los espacios. El programa fue ensamblado y simulado mediante Z80 Workbench.

1. Introducción

Un sistema basado en microprocesador requiere, además de la unidad de procesamiento, elementos de memoria e interfaces de entrada/salida que permitan almacenar programas, manejar información y comunicarse con dispositivos externos. Para integrar estos elementos es necesario establecer un mapa de direccionamiento y desarrollar la lógica de selección correspondiente, de manera que el microprocesador pueda acceder correctamente a cada dispositivo.

En esta actividad se desarrolla un sistema basado en el microprocesador Z80 que integra 16 KB de memoria para código, 16 KB de memoria para datos y seis puertos de entrada/salida. La memoria de código se ubica a partir de 0000H, la memoria de datos a partir de F800H y los puertos de entrada/salida a partir de 40H.

El diseño electrónico contempla la interconexión de los buses de direcciones y datos, así como las señales de control necesarias para acceder a las memorias y a las interfaces 8255. Debido a las características del mapa solicitado, se implementa un mecanismo de selección por bancos para acceder a los 16 KB físicos destinados a datos mediante una ventana ubicada entre F800H y FFFFH.

Como complemento del diseño electrónico, se desarrolla un programa en lenguaje ensamblador encargado de solicitar el nombre y apellidos del usuario, validar la información introducida, almacenarla y contabilizar la cantidad de letras sin considerar los espacios.

2. Funcionamiento del sistema

2.1. Microprocesador

El Z80 es un microprocesador de 8 bits que dispone de un bus de datos bidireccional de 8 bits (D0-D7) y un bus de direcciones de 16 bits (A0-A15). Por medio del bus de direcciones puede seleccionar hasta 65 536 posiciones, correspondientes a un espacio de direccionamiento de 64 KB comprendido entre 0000H y FFFFH.

En el sistema desarrollado, el Z80 constituye la unidad central encargada de ejecutar las instrucciones y controlar las transferencias de información entre las memorias y las interfaces de entrada/salida.

- A0-A15: bus de direcciones para indicar la posición de memoria o dirección de entrada/salida.
- D0-D7: bus bidireccional para transferir instrucciones y datos.
- /MREQ: identifica los accesos destinados a memoria.

- /IORQ: identifica las operaciones destinadas a las interfaces 8255.
- /RD: indica una operación de lectura.
- /WR: indica una operación de escritura.
- CLK: sincroniza las operaciones; en el diseño se empleó un oscilador de 4 MHz.
- /RESET: entrada activa en nivel bajo; después del reinicio la ejecución comienza en 0000H.
- /INT y /NMI: entradas de interrupción que se mantienen inactivas en este sistema.
- /WAIT: permite introducir estados de espera y se mantiene inactiva.
- /BUSREQ: permite solicitar el control de los buses y se mantiene inactiva.

Durante una operación, el Z80 coloca una dirección sobre A0-A15. Mediante /MREQ o /IORQ indica si el acceso corresponde a memoria o entrada/salida, mientras que /RD o /WR determinan si se realizará una lectura o una escritura. La lógica de decodificación habilita únicamente el dispositivo correspondiente.

2.2. Memorias

El sistema utiliza una EEPROM 28C256 destinada al almacenamiento del código y dos memorias SRAM 6264 destinadas al almacenamiento de datos.

La 28C256 es una memoria EEPROM organizada como 32K x 8 bits, equivalente a 32 KB. Dispone de 15 líneas de dirección (A0-A14) y 8 líneas de datos (D0-D7). Para este sistema se asignan 16 KB para código, comprendidos entre 0000H y 3FFFH. Entre sus principales señales de control se encuentran /CE, /OE y /WE.

Para la memoria de datos se utilizan dos SRAM 6264. Cada dispositivo presenta una organización de 8K x 8 bits, equivalente a 8 KB, por lo que ambas memorias proporcionan una capacidad física total de 16 KB. Cada 6264 dispone de 13 líneas de dirección (A0-A12) y 8 líneas de datos (D0-D7).

La memoria de datos debe comenzar en F800H. Sin embargo, el espacio comprendido entre F800H y FFFFH es de 0800H = 2048 bytes = 2 KB. Por esta razón, el diseño utiliza este intervalo como una ventana de acceso de 2 KB. Los 16 KB físicos se dividen en bancos que pueden presentarse dentro de la misma ventana.

Las líneas A0-A10 seleccionan una de las 2048 posiciones de la ventana. Las líneas PC0-PC2 del segundo 8255 participan en la

selección del banco, permitiendo seleccionar ocho regiones de 2 KB, para un total de 16 KB.

2.3. Interfaz para periféricos

Para implementar los seis puertos de entrada/salida se utilizan dos circuitos 8255 PPI (Programmable Peripheral Interface). Cada dispositivo dispone de 24 líneas programables organizadas en tres puertos de 8 bits: Puerto A, Puerto B y Puerto C.

A1	A0	Registro seleccionado
0	0	Puerto A
0	1	Puerto B
1	0	Puerto C
1	1	Registro de control

Los dispositivos se encuentran mapeados a partir de 40H. El primer 8255 ocupa las direcciones 40H-43H y el segundo las direcciones 44H-47H.

Dirección	Dispositivo	Registro
40H	U5	Puerto A
41H	U5	Puerto B
42H	U5	Puerto C
43H	U5	Registro de control
44H	U6	Puerto A
45H	U6	Puerto B
46H	U6	Puerto C
47H	U6	Registro de control

La comunicación entre el Z80 y los 8255 utiliza D0-D7, A0-A1 y las señales /CS, /RD, /WR y RESET. La señal /IORQ participa en la lógica de selección. En el diseño, PC0-PC2 del segundo 8255 se utilizan internamente para controlar la selección de bancos de la SRAM.

3. Diseño del sistema

3.1. Diagrama electrónico

El diagrama electrónico del sistema fue desarrollado mediante KiCad Schematic Capture. El diseño integra el microprocesador Z80, una EEPROM 28C256, dos memorias SRAM 6264, dos interfaces programables 8255 y los elementos lógicos necesarios para realizar la decodificación de direcciones y la generación de las señales de selección.

Dirección inicial	Dirección final	Tamaño	
0000H	3FFFFH	16 KB	Memoria de código
4000H	F7FFFH	-	Espacio no asignado
F800H	FFFFH	2 KB	Ventana de memoria de datos

Para la memoria de código se asignó el intervalo 0000H-3FFFFH: $3FFFFH - 0000H + 1 = 4000H = 16384 \text{ bytes} = 16 \text{ KB}$. El intervalo 4000H-F7FFFH permanece sin asignar. A partir de F800H se encuentra la ventana de 2 KB utilizada para acceder a la memoria de datos.

Debido a que los 16 KB físicos de SRAM no pueden ubicarse linealmente a partir de F800H dentro del espacio de direccionamiento del Z80, se utiliza selección por bancos. PC0-PC2 del segundo 8255 permiten seleccionar ocho bancos diferentes de 2 KB.

PC2	PC1	PC0	Banco	Tamaño
0	0	0	0	2 KB
0	0	1	1	2 KB
0	1	0	2	2 KB
0	1	1	3	2 KB
1	0	0	4	2 KB
1	0	1	5	2 KB
1	1	0	6	2 KB
1	1	1	7	2 KB

Para realizar la selección de los diferentes dispositivos se utilizaron decodificadores 74LS138, complementados con compuertas lógicas. En los accesos a memoria, /MREQ indica que la operación corresponde al espacio de memoria. Para los dispositivos de entrada/salida se utiliza /IORQ. Las señales /RD y /WR establecen el sentido de la transferencia.

El sistema utiliza un oscilador de 4 MHz para proporcionar CLK al Z80. También se implementa el circuito de reinicio necesario para inicializar el microprocesador y los 8255, considerando la polaridad requerida por cada dispositivo.

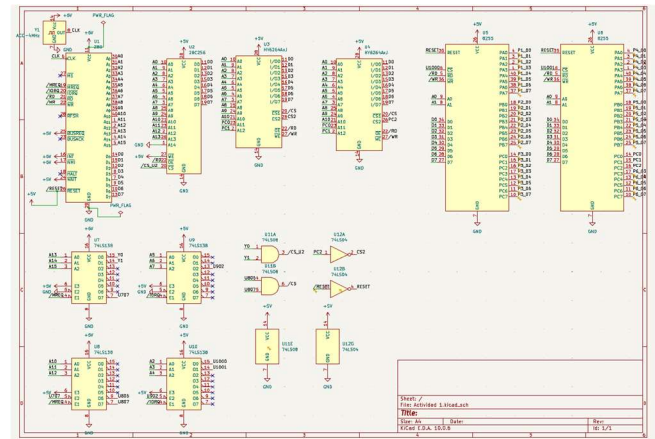


Figura 1: Diagrama electrónico del sistema basado en el microprocesador Z80.

Una vez completado el esquemático se utilizó la herramienta ERC (Electrical Rules Checker) de KiCad para revisar las conexiones realizadas y detectar posibles problemas eléctricos dentro del diagrama.

3.2 Mapa de memoria

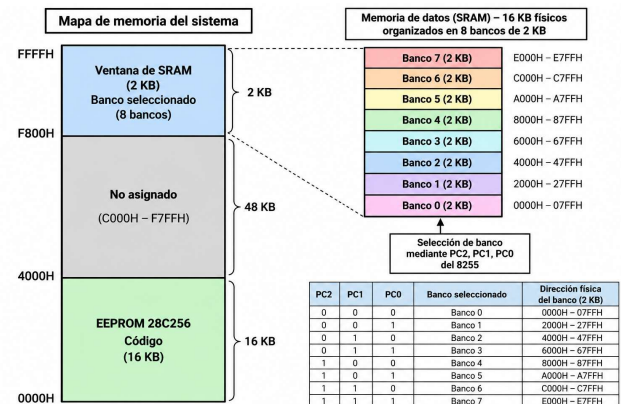


Figura 2: Mapa de memoria del sistema.

4. Análisis de los resultados

Como resultado del diseño se obtuvo el diagrama electrónico de un sistema basado en el microprocesador Z80 que integra memoria de código, memoria de datos e interfaces de entrada/salida. El hardware fue desarrollado mediante KiCad Schematic Capture y el programa en lenguaje ensamblador fue ensamblado y simulado mediante Z80 Workbench.

En el diseño electrónico se estableció la interconexión del Z80 con la EEPROM 28C256, las dos memorias SRAM 6264 y las dos interfaces 8255. Asimismo, se incorporó la lógica de decodificación necesaria para generar las señales de selección de los diferentes dispositivos a partir del bus de direcciones y las señales de control.

El intervalo 0000H-3FFFFH quedó destinado a los 16 KB de código, mientras que para los datos se estableció una ventana entre F800H y FFFFFH. Debido al tamaño de esta ventana, el acceso a los 16 KB físicos de SRAM se resolvió mediante ocho bancos de 2 KB. Los dispositivos de entrada/salida se distribuyeron entre 40H y 47H.

Una vez finalizadas las conexiones se realizó la revisión del esquemático mediante ERC. Esta herramienta permitió comprobar las reglas eléctricas y detectar problemas de conexión durante el desarrollo del circuito. La comprobación mediante ERC corresponde a una revisión del esquemático y no a una simulación funcional del hardware.

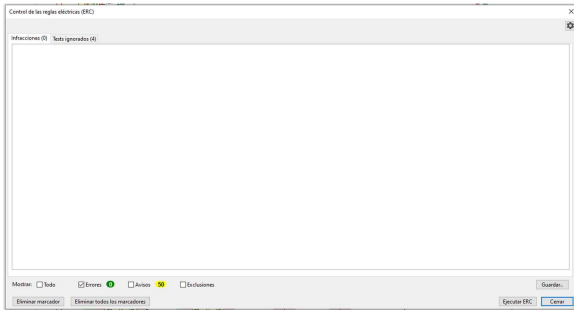


Figura 3: Comprobación del diagrama mediante ERC de KiCad.

Por otra parte, el programa desarrollado en lenguaje ensamblador fue probado mediante Z80 Workbench. Al iniciar la ejecución se muestra un mensaje solicitando el nombre y apellidos del usuario. Posteriormente, cada carácter introducido es evaluado para determinar si corresponde a una letra mayúscula, letra minúscula o espacio.

Para comprobar el funcionamiento del conteo se introdujo “Alexis Romero”. El programa aceptó correctamente las letras y el espacio existente entre el nombre y apellido. El resultado obtenido fue “NUMERO DE LETRAS: 12”. El valor es correcto debido a que “Alexis” contiene seis letras y “Romero” contiene seis, mientras que el espacio no incrementa el contador.



Figura 4: Prueba de captura y conteo de letras.

El programa utiliza F800H como dirección inicial para el almacenamiento de los caracteres aceptados, manteniendo correspondencia con la dirección inicial establecida para los datos en el diseño. Cada letra o espacio válido es almacenado, mientras que únicamente las letras incrementan el contador.

Para realizar la interacción con la terminal durante la simulación se utilizó el puerto 01H proporcionado por Z80 Workbench. Este puerto corresponde exclusivamente al entorno de simulación del programa y no representa las direcciones físicas 40H-47H asignadas a los 8255 en el diseño electrónico.

A partir de las pruebas realizadas se verificó que el programa solicita el nombre y apellidos, permite introducir espacios, acepta letras mayúsculas y minúsculas, detecta caracteres no válidos, contabiliza únicamente las letras y muestra el resultado obtenido.

5. Conclusión

Una de las principales consideraciones del diseño fue la ubicación de la memoria de datos a partir de F800H. Debido a que entre esta

dirección y FFFFH solamente existe un espacio de 2 KB, se implementó una ventana de direccionamiento junto con un mecanismo de selección por bancos para permitir el acceso a los 16 KB físicos disponibles en las memorias SRAM. Asimismo, la lógica de decodificación permitió establecer los rangos correspondientes para la memoria de código, la memoria de datos y las interfaces 8255.

El sistema diseñado corresponde a una arquitectura Von Neumann, ya que el Z80 utiliza un mismo espacio de direccionamiento y comparte los buses de direcciones y datos para acceder tanto al código como a los datos. Aunque se emplean dispositivos físicos diferentes para almacenarlos -una EEPROM para el programa y memorias SRAM para los datos-, ambos forman parte del espacio de memoria del microprocesador. Por otra parte, los dispositivos de entrada/salida utilizan un espacio independiente mediante la señal /IORQ.

Finalmente, el desarrollo del esquemático en KiCad permitió representar y revisar las conexiones necesarias entre los diferentes componentes, mientras que las pruebas realizadas en Z80 Workbench permitieron verificar el comportamiento del programa en lenguaje ensamblador. El programa cumplió con la captura del nombre y apellidos, la validación de caracteres y el conteo de letras sin considerar los espacios.

Referencias

- [1] Zilog, Inc., *Z80 CPU User Manual*, UM0080, sep. 2016. [En línea]. Disponible en: https://www.zilog.com/docs/z80/z80cpu_um.pdf. (Visitado: 20 sep. 2026).
- [2] Microchip Technology Inc., *AT28C256: Industrial Grade 256-Kbit (32,768 × 8) Paged Parallel EEPROM*, DS20006386B, 2022. [En línea]. Disponible en: <https://www1.microchip.com/downloads/aemDocuments/documents/MPD/ProductDocuments/DataSheets/AT28C256-Industrial-Grade-256-Kbit-Paged-Parallel-EEPROM-Data-Sheet-DS20006386.pdf>. (Visitado: 20 sep. 2026).
- [3] KiCad Documentation Contributors, *Schematic Editor, KiCad 9.0 Reference Manual*. [En línea]. Disponible en: <https://docs.kicad.org/9.0/es/eeschema/eeschema.html>. (Visitado: 20 sep. 2026).
- [4] Hitachi, Ltd., *IC Memory Data Book*, Hitachi America Ltd., 1988, pp. 125–132, “HM6264 Series: 8192-Word × 8-Bit High Speed CMOS Static RAM.” [En línea]. Disponible en: https://www.bitsavers.org/components/hitachi/_dataBooks/1988_M11_Hitachi_IC_Memory_Data_Book.pdf. (Visitado: 20 sep. 2026).
- [5] R. S. Gaonkar, *The Z80 Microprocessor: Architecture, Interfacing, Programming, and Design*, 2nd ed. Columbus, OH, USA: Merrill, 1992. [En línea]. Disponible en: <https://books.google.com/books?id=57NfQgAACAAJ>. (Visitado: 20 sep. 2026).
- [7] L. A. Leventhal, *Z80 Assembly Language Programming*. Berkeley, CA, USA: Osborne/McGraw-Hill, 1979. [En línea]. Disponible en: https://openlibrary.org/books/OL17306435M/Z80_assembly_language_programming. (Visitado: 20 sep. 2026).